

```
# Create room
sge.game.start_room = game.GameRoom(obj, background=background)
# Remove the mouse to increase performance by avoiding collision detection
sge.game.mouse.visible = False
sge.game.start_room.remove(sge.game.mouse)

# Here we go!
sge.game.start()
```

Initialiasing the session of the game with GameRoom

background layer as the background of the game.

Having done this much, we will initialize our participants in this game. The player and the invaders, respectively. We use the xrange function to loop over and create 4 identical invaders and create a singular player. Both of them are concatenated to an object in which the player comes first, followed by the invaders.

Thereafter, we initialize the session of the game calling the GameRoom function of the game class passing the above defined concatenated object (for the player and the invaders) and the background as parameters. It's better if we disable the mouse pointer when the game loads as well.

With this done, we have almost set up the environment for the game to be run on. Now, we shall have to add details and functionalities (and attributes) to our participants (players and the objects). This is something similar to what a default constructor does and is necessary for the steps involved in the genetic algorithm, because it is on the basis of the values of these genes that we'll be performing crossovers, mutation and checking fitness. We do that by setting the data members for the class **Invaders** with some values of our choice.

We achieve this by using the dictionary data structure in Python wherein the key value pairs stand for the attribute and its respective values. For each attribute or a gene, we have a minimum value, a maximum value and a random number that is generated by a gamma distribution. The boundary values (the maximum and the minimum) ensure that the value of a gene stays within that range and

if the value should exceed the upper limit or fall below the lower limit, the value is set to the boundary value. The keys for the dictionary (genes) we've used are **size**, **xvelocity** (velocity along x axis), **yvelocity** (velocity along y axis), the probability of changing direction along x axis and the probability of changing directions along the y axis.

In the init function, we assign the genes with values by generating them using the above approach. We also set a cool dynamic looking sprite around the player which adds to the visual aesthetics of the game play. Herein, we set the initial value of the fitness to zero which shall evolve over time.

We also use a function called **event_step** in which we increase the fitness by one. There is also the possibility of a direction switch and the invader bouncing off the walls.

```
gene_props = {
    'scale': {
        'min': 1,
        'max': 2,
        'gen': lambda: random.gammavariate(2, 0.5)+1
    },
    'alpha': {
        'min': 5,
        'max': 255,
        'gen': lambda: random.randint(20, 255)
    },
    'xvelocity': {
        'min': 0.01,
        'max': 5,
        'gen': lambda: random.gammavariate(2, 0.1)
    },
    'yvelocity': {
        'min': 0.01,
        'max': 5,
        'gen': lambda: random.gammavariate(2, 0.1)
    },
    'x_prob_change_dir': {
        'min': 0.00,
        'max': 0.05,
        'gen': lambda: random.uniform(0.0, 0.05)
    },
}
```

Setting the data members for Invaders

*CODING MATTERS



*Introduction to Genetic Algorithms: Theory and Applications

Learn the main mechanisms of Genetic Algorithm as a heuristic AI search or optimization in Matlab.. <https://dgit.in/GAUdemy>



*Genetic Algorithms

A video course from NPTEL taking you through the basics and advanced theory of genetic algorithms from IIT Madras <https://dgit.in/GANPTEL>



*Understanding Genetic Algorithms and Genetic Programming

This course teaches you how to implement two types of AI using simple C#. <https://dgit.in/GAPSight>